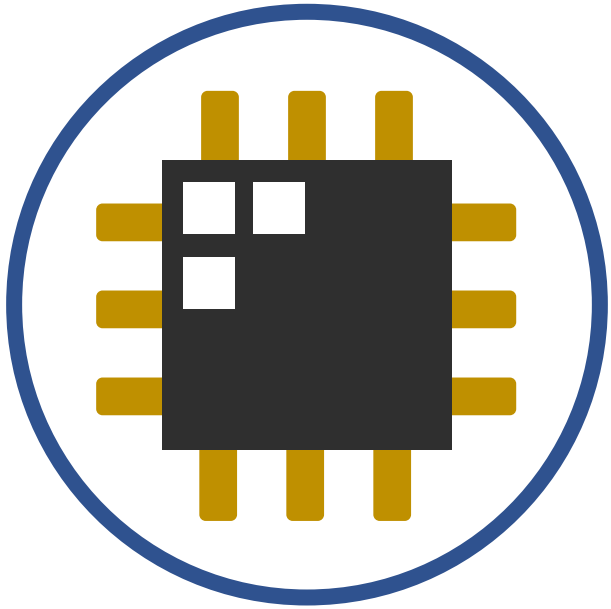




EEE 113:

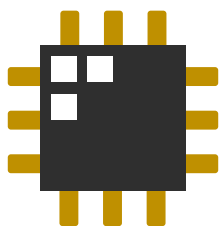
Introduction to Electrical and Electronics Engineering Systems

Week 4 – Assembly programming,
Hardware/Software Interface



- [illegible]

This is a *custom processor* made for EEE 113. Simple enough to be understood. Complex enough to implement simple programs. Previous EEE 113 offerings had ‘**M**achine **A**rchitecture that is **R**eally **I**ntuitive and **E**asy’ (MARIE)
Now, for this offering, we have ‘**A**rchitecture **D**esign that’s **E**asy to **L**earn’ (ADEL) ☺



Recap: Talking to your processor

- The instruction encoding that the processor supports

W	OPC	RS	DEST	SRC1	CONST/SRC2
↓	If W = 0: 00 – EQ 01 – NE 10 – LT 11 – GT If W = 1: 00 – ADD 01 – SUB 10 – AND 11 – OR	↓	If W = 1: 00 – R0 01 – R1 10 – R2 11 – R3 If W = 0: 00 – branches do not write	00 – R0 01 – R1 10 – R2 11 – R3	If RS = 0: The second operand is 8-bit CONST This is written in two's complement If RS = 1: Second operand is the last 2-bits SRC2 0000 00 <u>00</u> – R0 0000 00 <u>01</u> – R1 0000 00 <u>10</u> – R2 0000 00 <u>11</u> – R3
			If W = 0: 0 – branches have no second operand If W = 1: 0 – Constant second operand (ADDI/SUBI/ANDI/ORI) 1 – Register second operand (ADD/SUB/AND/OR)		

Sample instructions:

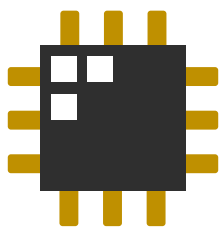
ADD R0, R2, R3

SUBI R3, R1, -31

BNE R2, -2

0: BEQ/BNE/BLT/BGT (Instructions that do not write to the register file)

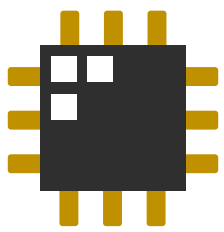
1: ADD/SUB/AND/OR and ADDI/SUBI/ANDI/ORI (Instructions that write to the register file)



Assembly programming

- The processor is complex enough to implement *cool* programs
 - Break down algorithms into instructions that the processor *understands*
 - Step-by-step instructions!

Let's have some assembly programming exercises!



Assembly programming example

- Multiplication through repeated addition: **R2 = R0 x R1**

ADDI R0, R0, 15

ADDI R1, R1, 7

BEQ R1, 4

loop:

ADD R2, R2, R0

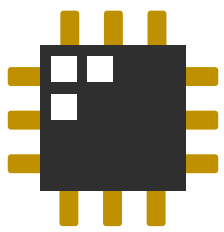
SUBI R1, R1, 1

BNE R1, -2

end:

BEQ R1, 0

What does each line do?



Assembly programming example

- Multiplication through repeated addition: **R2 = R0 x R1**

ADDI R0, R0, 15 *# first number to multiply*

ADDI R1, R1, 7 *# second number to multiply*

BEQ R1, 4 *# end, you're multiplying with 0*

loop:

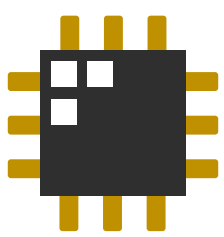
ADD R2, R2, R0 *# repeated addition, R2 gets the product*

SUBI R1, R1, 1 *# reduce this until it reaches 0*

BNE R1, -2 *# back to loop*

end:

BEQ R1, 0 *# infinite loop to halt program*



Assembly programming example

- Multiplication through repeated addition: **R2 = R0 x R1**

ADDI R0, R0, 15

ADDI R1, R1, 7

BEQ R1, 4

loop:

ADD R2, R2, R0

SUBI R1, R1, 1

BNE R1, -2

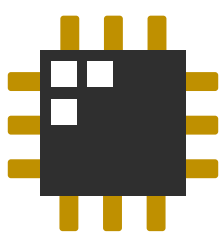
end:

BEQ R1, 0

(0x800F)
0x8507
0x0104
0x9A00
0xA501
0x21FE
0x0100

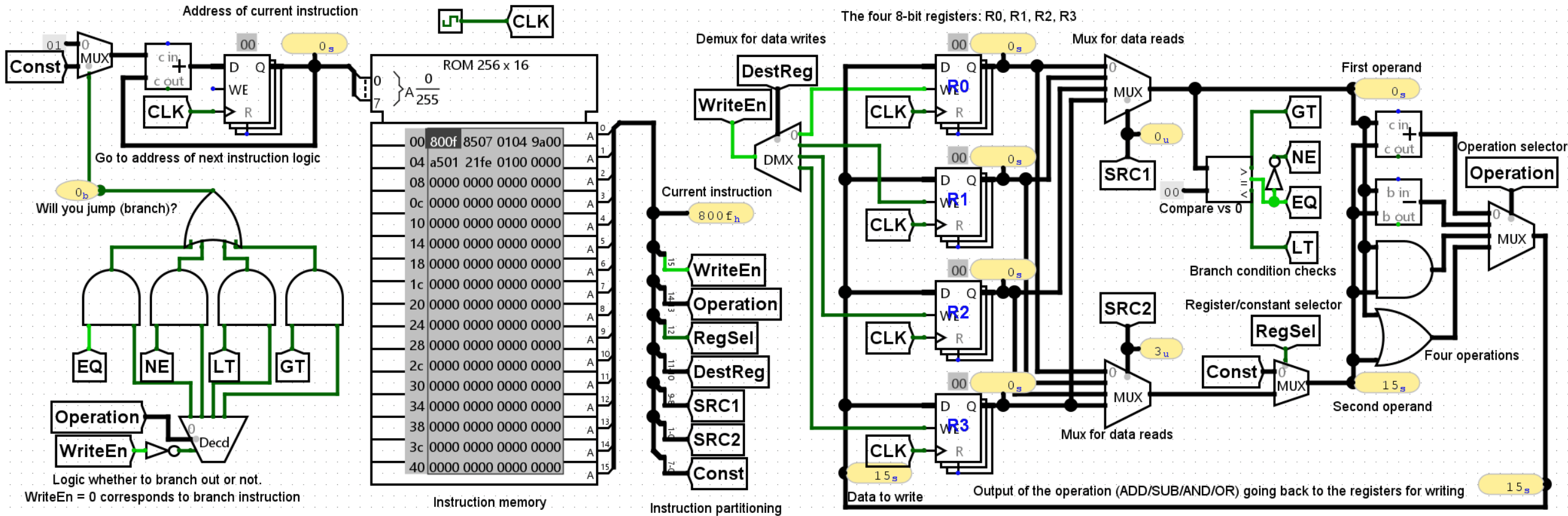
*The processor only
understands 0s and 1s!*

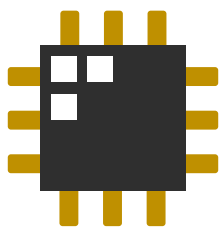
You can use the
Python assembler to
convert instructions
into machine code!



Assembly programming example

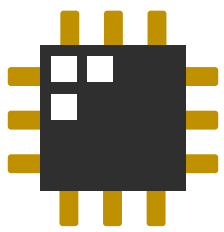
- Multiplication through repeated addition: **R2 = R0 x R1**





Assembly programming exercise 1

- Write an assembly program to perform **$R0 / R1 = R2$ remainder $R0$**
 - Assume **$R1$** will never be 0



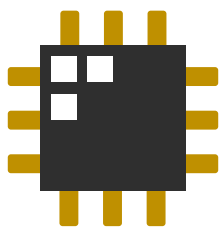
Assembly programming exercise 1

- Write an assembly program to perform **$R0 / R1 = R2$ remainder $R0$**
 - Assume **$R1$** will never be 0

Try writing a pseudocode on what to do step-by-step:

- 1) Put operands on $R0$ and $R1$
- 2) Create a loop, check if $R0 < R1$ to exit
- 3) Inside the loop: increment $R2$, $R0 = R0 - R1$
- 4) *Create an infinite loop at the 'exit' to halt the program*

(not really required, but it's a nice practice to 'stop' the processor for now)



Assembly programming exercise 1

- Write an assembly program to perform **$R0 / R1 = R2$ remainder $R0$**

$R0$ = numerator

$R1$ = denominator

loop:

if ($R0 < R1$), go to *exit*

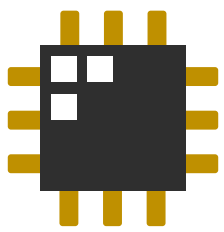
else, $R2 = R2 + 1$

$R0 = R0 - R1$

Go back to *loop*

***exit:* infinite loop here**

Convert to assembly!



Assembly programming exercise 1

- Write an assembly program to perform **$R0 / R1 = R2$ remainder $R0$**

ADDI R0, R0, 17

R0 = numerator

ADDI R1, R1, 5

R1 = denominator

loop:

SUB R3, R0, R1

Check if $R0 < R1$

BLT R3, exit

Go to exit if indeed $R0 < R1$

ADDI R2, R2, 1

Increment R2

SUB R0, R0, R1

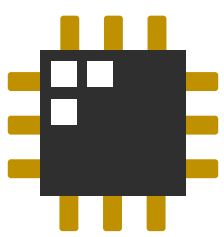
$R0 = R0 - R1$

BGT R1, loop

Go back to loop

exit: **BLT R3, exit**

infinite loop to 'stop' program



Assembly programming exercise 1

- Write an assembly program to perform **$R0 / R1 = R2$ remainder $R0$**

ADDI R0, R0, 17

R0 = numerator

ADDI R1, R1, 5

R1 = denominator

loop:

SUB R3, R0, R1

Check if $R0 < R1$

BLT R3, 4

Go to exit if indeed $R0 < R1$

ADDI R2, R2, 1

Increment R2

SUB R0, R0, R1

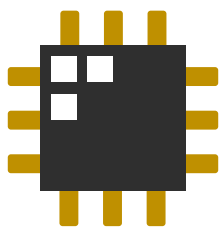
$R0 = R0 - R1$

BLT R1, -4

Go back to loop

exit: **BLT R3, 0**

infinite loop to 'stop' program



Assembly programming exercise 1

- Write an assembly program to perform **R0 / R1 = R2 remainder R0**

ADDI R0, R0, 17 *0x8011*

ADDI R1, R1, 5 *0x8505*

loop:

SUB R3, R0, R1 *0xBC01*

BLT R3, 4 *0x4304*

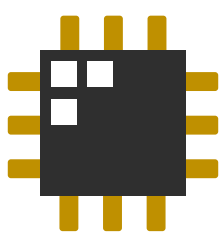
ADDI R2, R2, 1 *0x8A01*

SUB R0, R0, R1 *0xB001*

BGT R1, -4 *0x61FC*

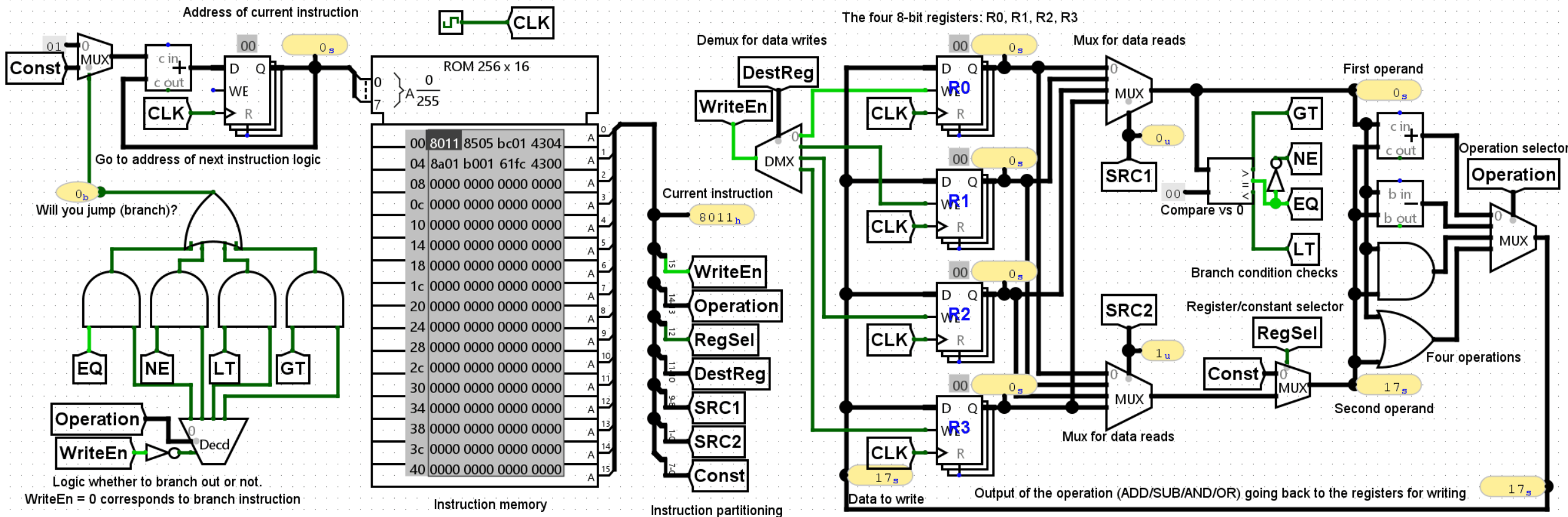
exit: **BLT R3, 0** *0x4300*

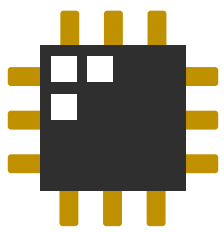
Let's run this now!



Assembly programming exercise 1

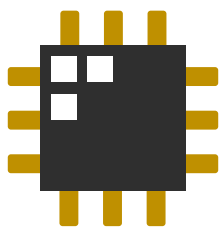
- Write an assembly program to perform $R0 / R1 = R2 \text{ remainder } R0$





Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

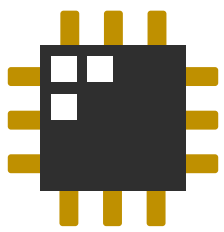


Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

Try writing a pseudocode on what to do step-by-step:

- 1) Increment R3: $R3 = R3 + 1$
- 2) If $R3 \neq 9$, go back to step 1. Otherwise, proceed.
- 3) Set $R3 = 0$, then go back to 1.



Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

loop:

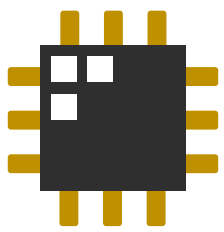
R3 = R3 + 1

if (R3 != 9), go to *loop*

R3 = 0

go to *loop*

Convert to assembly!



Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

loop:

ADDI R3, R3, 1

Increment R3

SUBI R2, R3, 9

Check if R3 != 9

BNE R2, *loop*

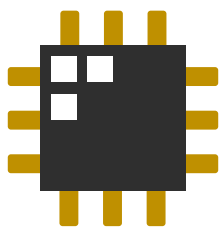
If it is, go back to loop

SUB R3, R3, R3

Else, set R3 = 0

BEQ R3, *loop*

Go back to loop



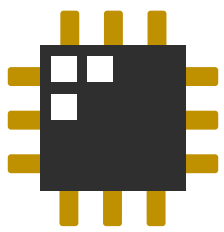
Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

loop:

```
ADDI R3, R3, 1
SUBI R2, R3, 9
BNE R2, [-2]
SUB R3, R3, R3
BEQ R3, [-4]
```

```
# Increment R3
# Check if R3 != 9
# If it is, go back to loop
# Else, set R3 = 0
# Go back to loop
```



Assembly programming exercise 2

- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)

loop:

ADDI R3, R3, 1

0x8F01

SUBI R2, R3, 9

0xAB09

BNE R2, -2

0x22FE

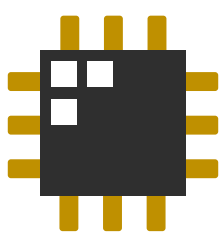
SUB R3, R3, R3

0xBF03

BEQ R3, -4

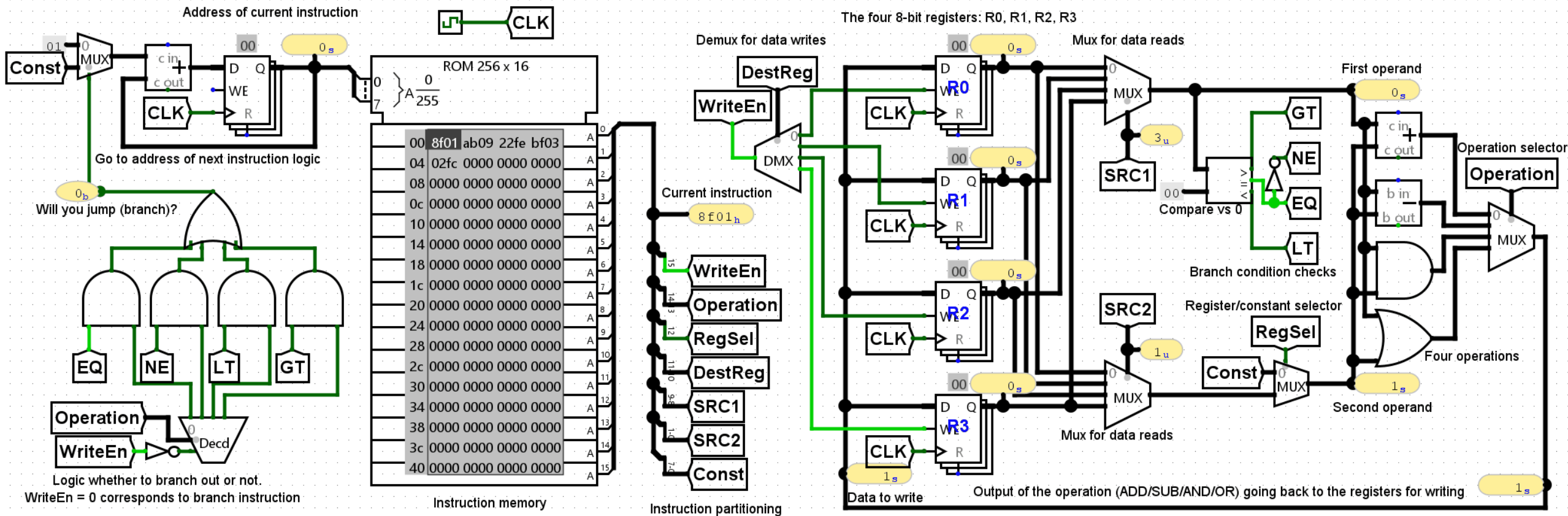
0x03FC

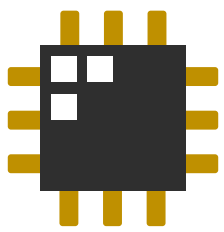
Let's run this now!



Assembly programming exercise 2

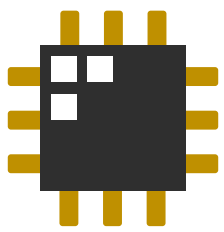
- Write an assembly program to increment R3 from 0 to 9, then back to 0
 - This is an infinite loop. R3 just counts up then loops back to 0 (0000) after reaching 9 (1001)





Assembly programming exercise 3

- Write the assembly code that gives out the n th Fibonacci number (stored in R0) and stores the result in R3.

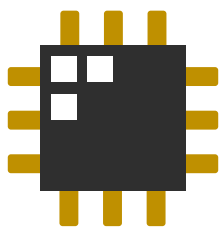


Assembly programming exercise 3

- Write the assembly code that gives out the n th Fibonacci number (stored in R0) and stores the result in R3.

Try writing a pseudocode on what to do step-by-step:

- 1) Set $R0 = n$, $R1 = 0$, $R2 = 1$, $R3 = R2$ (1st Fibonacci number)
- 2) Decrement R0. Then, check if $R0 == 0$, if true, exit prog
- 3) $R3 = R1 + R2$
- 4) $R1 = R2$
- 5) $R2 = R3$
- 6) Loop back to step 2



Assembly programming exercise 3

- Write the assembly code that gives out the n th Fibonacci number (stored in R0) and stores the result in R3.

R0 = R0 + n (say 8)

R2 = R2 + 1

R3 = R2

loop:

R0 = R0 - 1

if (R0 == 0), go to *exit*

else, R3 = R1 + R2

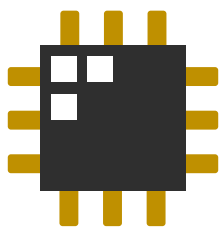
R1 = R2

R2 = R3

go to *loop*

***exit:* end of program (inf loop)**

Convert to assembly!



Assembly programming exercise 3

ADDI R0, R0, 8

Initialize R0, $n = 8$

ADDI R2, R2, 1

$R2 = 1$

ADDI R3, R2, 0

$R3 = R2$

loop:

SUBI R0, R0, 1

Decrement R0

BEQ R0, exit

Check if $R0 == 0$; if it is, exit

ADD R3, R1, R2

$R3 = R1 + R2$ (Fibonacci)

ADDI R1, R2, 0

$R1 = R2$ (adjust 1st term)

ADDI R2, R3, 0

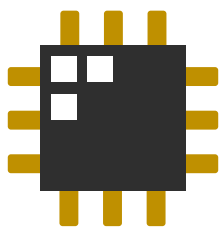
$R2 = R3$ (adjust 2nd term)

BNE R0, loop

loop back to loop label

***exit:* BEQ R0, exit**

infinite loop to 'halt' program



Assembly programming exercise 3

ADDI R0, R0, 8

Initialize R0, n = 8

ADDI R2, R2, 1

R2 = 1

ADDI R3, R2, 0

R3 = R2

loop:

SUBI R0, R0, 1

Decrement R0

BEO R0, 5

Check if R0 == 0; if it is, exit

ADD R3, R1, R2

R3 = R1 + R2 (Fibonacci)

ADDI R1, R2, 0

R1 = R2 (adjust 1st term)

ADDI R2, R3, 0

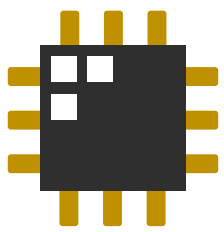
R2 = R3 (adjust 2nd term)

BNE R0, -5

loop back to loop label

exit: **BEO R0, 0**

infinite loop to 'halt' program



Assembly programming exercise 3

ADDI R0, R0, 8 *0x8008*

ADDI R2, R2, 1 *0x8A01*

ADDI R3, R2, 0 *0x8E00*

loop:

SUBI R0, R0, 1 *0xA001*

BEQ R0, 5 *0x0005*

ADD R3, R1, R2 *0x9D02*

ADDI R1, R2, 0 *0x8600*

ADDI R2, R3, 0 *0x8B00*

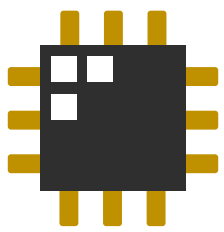
BNE R0, -5 *0x20FB*

exit: **BEQ R0, 0** *0x0000*

Let's run this now!



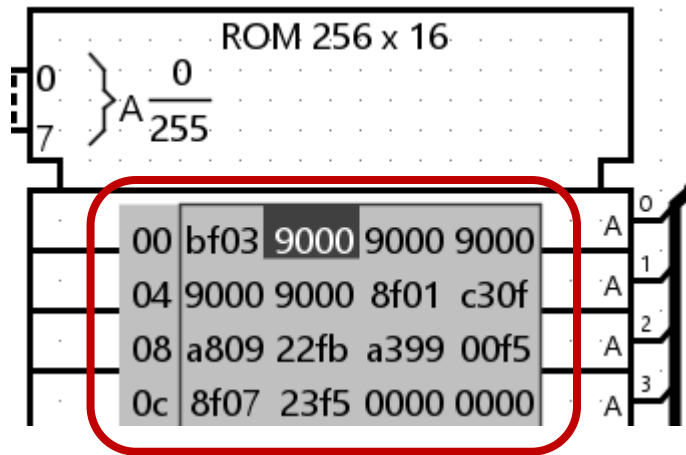
-
- Address of current instruction
- Const
- MUX
- c in
- c out
- CLK
- D
- Q
- WE
- R
- 0_s
- ROM 256 x 16
- 0 } A 0
- 255
- 00 8008 8a01 8e00 a001
- 04 0005 9d02 8600 8b00
- 08 20fb 0000 0000 0000
- 0c 0000 0000 0000 0000
- 10 0000 0000 0000 0000
- 14 0000 0000 0000 0000
- 18 0000 0000 0000 0000
- 1c 0000 0000 0000 0000
- 20 0000 0000 0000 0000
- 24 0000 0000 0000 0000
- 28 0000 0000 0000 0000
- 2c 0000 0000 0000 0000
- 30 0000 0000 0000 0000
- 34 0000 0000 0000 0000
- 38 0000 0000 0000 0000
- 3c 0000 0000 0000 0000
- 40 0000 0000 0000 0000
- 0 } A 0
- 255
- Current instruction
- 8008_h
- WriteEn
- Operation
- RegSel
- DestReg
- SRC1
- SRC2
- Const
- Demux for data writes
- WriteEn
- DMX
- CLK
- R0
- R1
- R2
- R3
- Mux for data reads
- MUX
- SRC1
- 0_u
- Compare vs 0
- GT
- NE
- EQ
- LT
- First operand
- 0_s
- c in
- c out
- b in
- b out
- Operation selector
- Operation
- MUX
- Branch condition checks
- Register/constant selector
- RegSel
- Const
- MUX
- Second operand
- 8_s
- Mux for data reads
- MUX
- Const
- 0_u
- Output of the operation (ADD/SUB/AND/OR) going back to the registers for writing
- 8_s
- Logic whether to branch out or not.
- WriteEn = 0 corresponds to branch instruction
- Will you jump (branch)?
- EQ
- NE
- LT
- GT
- Operation
- WriteEn
- Decd
- Go to address of next instruction logic



The hardware/software interface

- In the past, machine language was used
 - Imagine just writing a program using only binary/hex!

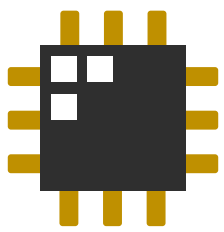
This can be too tedious!



Imagine just writing this...



Computer scientist Margaret Hamilton poses with the Apollo guidance software that she and her team developed at MIT. Photos: MIT Museum



The hardware/software interface

- Eventually, symbolic notation was developed
 - An assembler translates assembly language to machine language
 - The assembler also computes for the relative offsets (branches for example!)
- However, assembly programming ***was still tedious*** since programmers have to think at the *machine level*
 - Step-by-step instructions
 - Code tends to be very long!
 - Syntax also makes the code hard to read the longer it becomes...

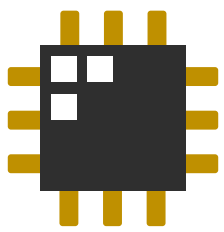
Assembly
language
program
(for MIPS)

```
swap:
    muli $2, $5, 4
    add  $2, $4, $2
    lw   $15, 0($2)
    lw   $16, 4($2)
    sw   $16, 0($2)
    sw   $15, 4($2)
    jr   $31
```

Assembler

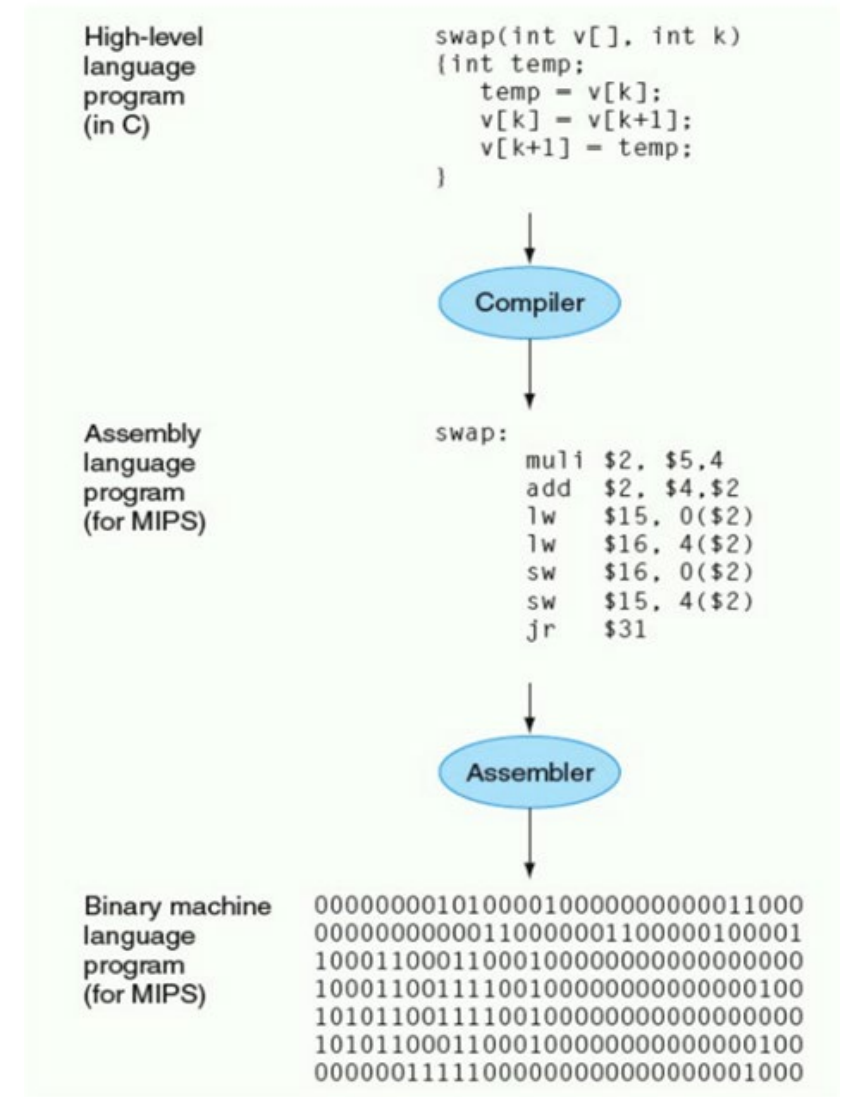
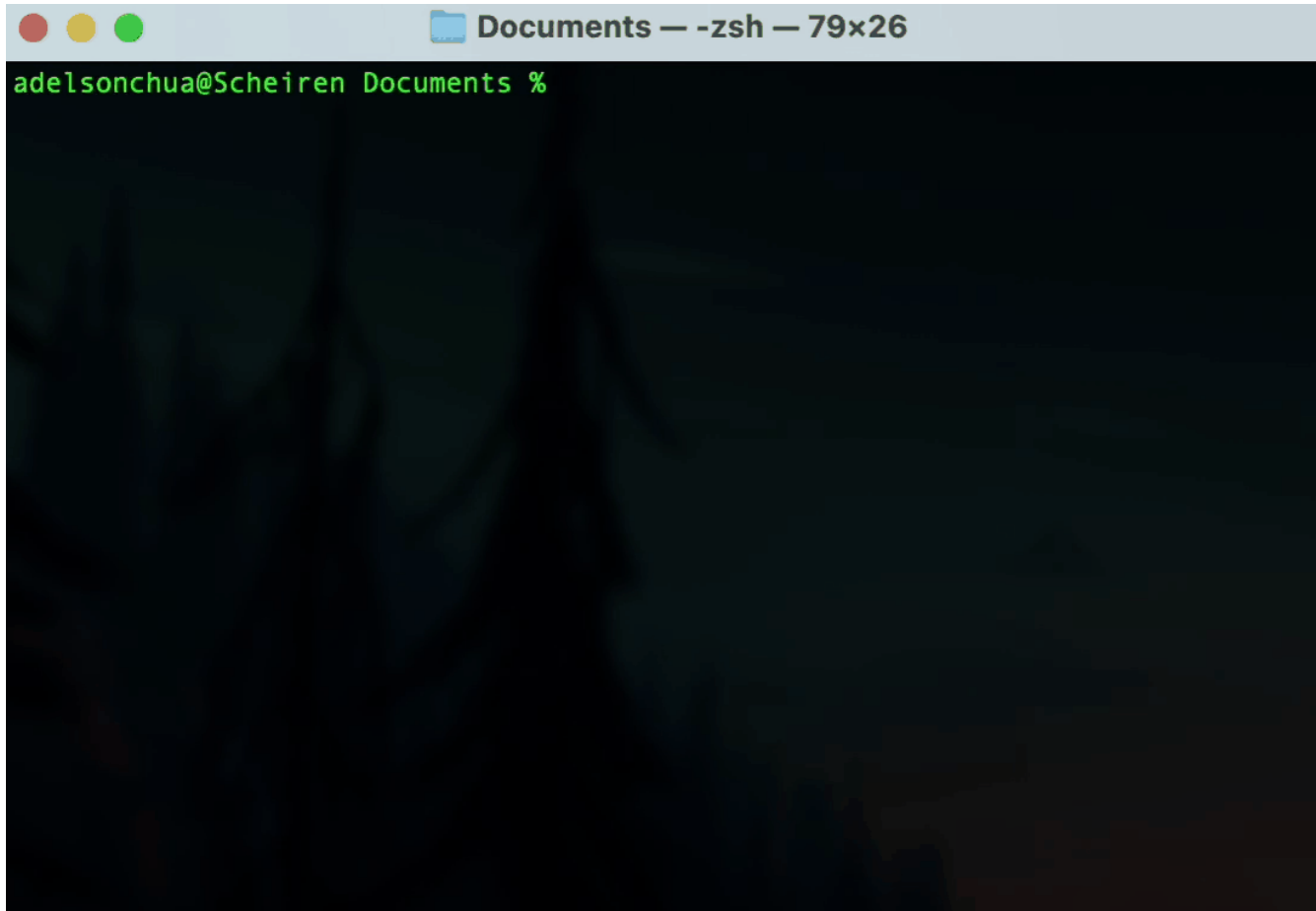
Binary machine
language
program
(for MIPS)

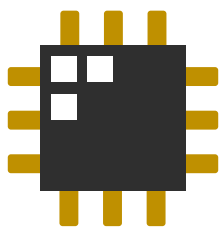
```
000000001010000100000000000011000
000000000000110000001100000100001
100011000110001000000000000000000
100011001111001000000000000000100
101011001111001000000000000000000
101011000110001000000000000000100
00000011111000000000000000001000
```



The hardware/software interface

- Compilers were later developed allowing the use of high-level language to write software





The hardware/software interface

- We're currently at the transition point now! 😊

